

Compliance & Secure Coding Practices

Compliance & Secure Coding Practices in 25-26J-041

Project ID: **25-26J-041**

Project Title:

The Guardian: A Modular AI for Proactive Human-Layer Cybersecurity
on Social Media

Student Details:

Names: Herath, H. M. G. D. K. (IT22361486)

De Alwis, H. P. G. H. S. (IT22317926)

Fernando, M. R. I. (IT22366054)

Vishwa, J. R. (IT22916112)

Supervisor: Mr. Kavinga Yapa Abeywardena

Co-supervisor: Mr. Deemantha Siriwardana

Date of Submission: 30 April 2026

1. Introduction

The Guardian is a modular, AI-powered cybersecurity system designed to provide proactive protection against human-layer threats on social media platforms. Implemented as a browser extension with a FastAPI-based backend, it integrates Natural Language Processing (NLP) modules to detect misinformation, hate speech, phishing/scams, and privacy risks such as Personally Identifiable Information (PII) and Operational Security (OpSec) exposure. Particular emphasis is placed on the PII and OpSec module, which models inference-based privacy risks through attacker-aware reasoning.

Due to the sensitive nature of user-generated content, compliance with security standards and secure coding practices is essential. The system incorporates privacy-by-design, stateless processing, and secure API communication to ensure data protection, system integrity, and ethical AI deployment.

2. Secure Coding Standards

The development of The Guardian adheres to established secure coding standards to ensure confidentiality, integrity, and availability. OWASP Secure Coding Practices guided the design of both the browser extension and backend API, including strict input validation, secure data handling, prevention of injection vulnerabilities, and enforcement of HTTPS-based communication. API interactions are restricted to structured JSON formats, minimizing risks from malformed or malicious inputs.

Additionally, principles from ISO/IEC 27001 were incorporated to align with a secure Software Development Lifecycle (SDLC). This is reflected in the system's privacy-by-design approach, stateless processing, modular architecture, anonymized request handling, and encrypted communication. While CERT standards are not directly applicable, the system emphasizes secure design, risk minimization, and resilience against web-based and human-layer threats.

3. Key Secure Coding Techniques Implemented with Examples

Several secure coding techniques were implemented to enhance system security and reduce vulnerabilities.

Input Validation:

All inputs are validated using a structured JSON schema with constraints such as input length (1–5000 characters) and UTF-8 encoding. This prevents malformed or malicious inputs and reduces injection risks.

```
{  
  "textToAnalyze": "string (1–5000 chars)",  
  "context": { "anonymousUserId": "UUIDv4" }  
}
```

Error Handling:

Structured error handling uses standardized HTTP response codes (400, 429, 500). Error messages provide necessary debugging information without exposing sensitive system details, preventing information leakage.

Authentication & Authorization:

Instead of traditional authentication, the system uses anonymized UUID-based identifiers. This eliminates credential storage, reduces attack surfaces, and prevents unauthorized access risks.

Encryption:

All communication between the browser extension and backend API is secured using HTTPS (TLS), protecting data from interception and Man-in-the-Middle (MitM) attacks.

Code Access Control:

The browser extension operates within a sandboxed Web Extension environment, preventing direct OS access. Backend operations are restricted to controlled API endpoints, ensuring secure execution.

In the PII and OpSec module, secure processing is further enhanced through an attacker-aware design approach. Instead of processing signals independently, the system evaluates how combinations of personal identifiers and contextual cues may expose users to real-world risks. This includes detecting patterns such as future travel plans, absence from home, and behavioral routines. These signals are processed using a hybrid pipeline combining rule-based validation and machine learning models, ensuring both precision and contextual understanding while maintaining secure data handling.

4. Tools & Technologies Used for Secure Coding

The Guardian utilizes modern tools to enforce secure coding practices across development stages. The backend is implemented using FastAPI (Python), supporting schema validation and reducing malformed input risks, while Hugging Face Transformers and PyTorch enable controlled NLP model execution. Data processing is handled using NumPy and Pandas for structured input handling.

Security practices were integrated across the SDLC, where OWASP principles guided design, secure coding practices were applied during implementation, and OWASP ZAP was used during testing to identify vulnerabilities such as insecure endpoints and improper input handling. During deployment, HTTPS communication was enforced to secure data transmission.

Additionally, GitHub was used for version control and code review. Reviews focused on input validation, API security, error handling, and data protection, with identified issues resolved before integration to ensure code quality and compliance.

5. Compliance Standards

The Guardian system aligns with the principles of the General Data Protection Regulation

(GDPR) and ISO/IEC 27001 to ensure secure and privacy-preserving data handling. GDPR requirements such as data minimization and privacy-by-design are achieved through a stateless architecture, where user-generated content is processed in real time without storage or logging. User identity is protected through anonymized UUIDs, eliminating personal data collection and reducing exposure risks. All communication is secured via HTTPS, ensuring encrypted data transmission.

The PII and OpSec module further strengthens GDPR alignment by minimizing sensitive data exposure through real-time processing and avoiding persistent storage. Potentially sensitive contextual inferences are analyzed transiently, supporting privacy-by-design principles.

In line with ISO/IEC 27001, the system incorporates secure architecture, controlled data processing, and risk-aware design. Supporting documentation such as the SRS and API contract ensures transparency and traceability. User consent is obtained through browser extension permissions, where users are clearly informed about data access and its purpose. The system does not perform background tracking and only analyzes visible content, ensuring user awareness and control.

6. Ethical Considerations

The Guardian system incorporates ethical considerations by emphasizing user privacy, transparency, and autonomy. It follows a non-intrusive design approach, providing informative “nudges” rather than restricting or censoring content, allowing users to make informed decisions independently. Fairness is also considered in AI-based detection to minimize bias and reduce false positives, particularly in sensitive areas such as hate speech analysis.

From a compliance perspective, the system aligns with GDPR and ISO/IEC 27001 principles through privacy-by-design, anonymized data handling, and secure communication protocols. These measures ensure ethical responsibility while maintaining regulatory compliance, user trust, and system effectiveness.

7. Challenges & Mitigation Strategies

A key challenge in implementing secure coding practices was balancing real-time performance with strong privacy and security requirements. Ensuring low-latency processing (<500 ms) while maintaining secure data handling was addressed through a stateless architecture, eliminating storage, and reducing exposure risks.

A key challenge in the PII and OpSec module was balancing accurate contextual inference with privacy preservation, as over-detection could impact usability while under-detection could expose users to risks. This was mitigated through calibrated signal weighting and hybrid detection strategies combining rule-based validation with machine learning models.

Additionally, secure communication was ensured by enforcing HTTPS-based API interactions, enabling encryption without increasing system complexity. These strategies enabled a balance between usability, performance, and security compliance.

8. References

- [1] OWASP Foundation, “OWASP Secure Coding Practices.” [Online]. Available: <https://owasp.org>
- [2] OWASP Foundation, “OWASP Zed Attack Proxy (ZAP).” [Online]. Available: <https://owasp.org/www-project-zap/>
- [3] ISO/IEC, “ISO/IEC 27001: Information Security Management Systems,” 2022.
- [4] European Union, “General Data Protection Regulation (GDPR).” [Online]. Available: <https://gdpr.eu>
- [5] Carnegie Mellon University, “CERT Secure Coding Standards.” [Online]. Available: <https://www.cert.org/secure-coding/>
- [6] FastAPI, “FastAPI Documentation.” [Online]. Available: <https://fastapi.tiangolo.com>
- [7] Hugging Face, “Transformers Documentation.” [Online]. Available: <https://huggingface.co/docs/transformers>
- [8] PyTorch, “PyTorch Documentation.” [Online]. Available: <https://pytorch.org/docs/>